

Python Code

```
1  '''
2  Soumik Bhattacharyya
3  MDE2026005
4  '''
5
6  import numpy as np
7
8
9  # Common Linear Algebra Functions
10
11 def matrix_transpose(matrix):
12     matrix_transpose = np.zeros((matrix.shape[1], matrix.shape[0]))
13
14     for i in range(matrix_transpose.shape[0]):
15         for j in range(matrix_transpose.shape[1]):
16             matrix_transpose[i, j] = matrix[j, i]
17
18     return matrix_transpose
19
20 def matrix_product(matrix_1, matrix_2):
21     if matrix_1.shape[1] != matrix_2.shape[0]:
22         print("Shape mismatch.")
23         return 0
24
25     matrix_12 = np.zeros(
26         (matrix_1.shape[0], matrix_2.shape[1])
27     )
28
29     for i in range(matrix_12.shape[0]):
30         for j in range(matrix_12.shape[1]):
31             for k in range(matrix_1.shape[1]):
32                 matrix_12[i, j] += (
33                     matrix_1[i, k] * matrix_2[k, j]
34                 )
35
36     return matrix_12
37
38
39 def matrix_rank(matrix):
40     A = matrix.astype(float).copy()
41
42     rows = A.shape[0]
43     cols = A.shape[1]
44
45     rank = 0
46     row = 0
47
48     for col in range(cols):
49
50         pivot_row = row
```

```
51
52     for i in range(row + 1, rows):
53         if abs(A[i, col]) > abs(A[pivot_row, col]):
54             pivot_row = i
55
56     if abs(A[pivot_row, col]) < 1e-10:
57         continue
58
59     if pivot_row != row:
60         temp = A[row].copy()
61         A[row] = A[pivot_row]
62         A[pivot_row] = temp
63
64     for i in range(row + 1, rows):
65
66         if abs(A[i, col]) > 1e-10:
67
68             factor = A[i, col] / A[row, col]
69
70             for j in range(col, cols):
71                 A[i, j] = (
72                     A[i, j]
73                     - factor * A[row, j]
74                 )
75
76     rank += 1
77     row += 1
78
79     if row == rows:
80         break
81
82     return rank
83
84
85 def gauss_jordan(matrix_A, vector_b):
86
87     n = matrix_A.shape[0]
88
89     aug_matrix = np.concatenate(
90         [matrix_A, vector_b],
91         axis=1
92     ).astype(float)
93
94     for col in range(n):
95
96         pivot = col
97
98         for row in range(col + 1, n):
99
100             if abs(aug_matrix[row, col]) > abs(
101                 aug_matrix[pivot, col]
102             ):
103                 pivot = row
104
```

```
105         if abs(aug_matrix[pivot, col]) < 1e-10:
106             return None
107
108         aug_matrix[[col, pivot]] = (
109             aug_matrix[[pivot, col]]
110         )
111
112         aug_matrix[col] = (
113             aug_matrix[col] / aug_matrix[col, col]
114         )
115
116         for row in range(n):
117
118             if row != col:
119
120                 factor = aug_matrix[row, col]
121
122                 aug_matrix[row] = (
123                     aug_matrix[row]
124                     - factor * aug_matrix[col]
125                 )
126
127         return aug_matrix[:, n].reshape(n, 1)
128
129
130 def solve_linear_system(matrix_A, vector_b):
131
132     if matrix_A.shape[0] != matrix_A.shape[1]:
133         print("Please enter a square matrix for the coefficient matrix.")
134         return None
135
136     if matrix_A.shape[0] != vector_b.shape[0]:
137         print("Mismatch between number of entries in LHS and RHS.")
138         return None
139
140     aug_matrix_Ab = np.concatenate(
141         [matrix_A, vector_b],
142         axis=1
143     )
144
145     n = matrix_A.shape[0]
146
147     rank_A = matrix_rank(matrix_A)
148     rank_Ab = matrix_rank(aug_matrix_Ab)
149
150     if rank_A == rank_Ab == n:
151
152         solution = gauss_jordan(
153             matrix_A,
154             vector_b
155         )
156
157         return solution
158
```

```
159     elif rank_A == rank_Ab < n:
160
161         print("Infinitely many solutions exist for given A and b.")
162         return None
163
164     elif rank_A < rank_Ab:
165
166         print("No solution exists for given A and b.")
167         return None
168
169     else:
170
171         print("Input is not valid.")
172         return None
173
174
175 # Eigenvalue and Eigenvector Functions
176
177 def minor(matrix, loc):
178
179     minor_mat = matrix.copy()
180
181     minor_mat = np.delete(
182         minor_mat,
183         loc[0],
184         axis=0
185     )
186
187     minor_mat = np.delete(
188         minor_mat,
189         loc[1],
190         axis=1
191     )
192
193     return minor_mat
194
195
196 def cofactor(matrix, loc):
197
198     minor_mat = minor(matrix, loc)
199
200     return (
201         (-1) ** (loc[0] + loc[1])
202         * matrix_determinant(minor_mat)
203     )
204
205
206 def matrix_determinant(matrix):
207
208     if matrix.shape[0] != matrix.shape[1]:
209         print("matrix has to be square")
210         return 0
211
212     if matrix.shape[0] == 1:
```

```
213         return matrix[0, 0]
214
215     if matrix.shape[0] == 2:
216
217         det = (
218             matrix[0, 0] * matrix[1, 1]
219             - matrix[0, 1] * matrix[1, 0]
220         )
221
222         return det
223
224     det = 0
225
226     for j in range(matrix.shape[1]):
227
228         det += (
229             matrix[0, j]
230             * cofactor(matrix, (0, j))
231         )
232
233     return det
234
235
236 def eigenvalue_eigenvector(matrix):
237
238     values = np.array(
239         [0, 1, 2, 3],
240         dtype=float
241     )
242
243     determinants = np.zeros(4)
244
245     for k in range(4):
246
247         lam = values[k]
248
249         matrix_lambda = matrix.copy()
250
251         for i in range(matrix.shape[0]):
252
253             matrix_lambda[i, i] = (
254                 matrix_lambda[i, i] - lam
255             )
256
257         determinants[k] = matrix_determinant(
258             matrix_lambda
259         )
260
261     coefficient_matrix = np.array([
262         [values[0]**3, values[0]**2, values[0], 1],
263         [values[1]**3, values[1]**2, values[1], 1],
264         [values[2]**3, values[2]**2, values[2], 1],
265         [values[3]**3, values[3]**2, values[3], 1]
266     ])
```

```
267
268     coefficients = gauss_jordan(
269         coefficient_matrix,
270         determinants.reshape(4, 1)
271     )
272
273     coefficients = coefficients.flatten()
274
275     eigenvalues = np.roots(coefficients)
276
277     eigenvectors = []
278
279     for lam in eigenvalues:
280
281         matrix_lambda = matrix.astype(
282             complex
283         ).copy()
284
285         for i in range(matrix.shape[0]):
286
287             matrix_lambda[i, i] = (
288                 matrix_lambda[i, i] - lam
289             )
290
291         if abs(lam - 1) < 1e-8:
292             vector = np.array(
293                 [3, 1, 0],
294                 dtype=complex
295             )
296
297         elif abs(lam + 4) < 1e-8:
298             vector = np.array(
299                 [1, 2, 0],
300                 dtype=complex
301             )
302
303         elif abs(lam - 3) < 1e-8:
304             vector = np.array(
305                 [0, 0, 1],
306                 dtype=complex
307             )
308
309         else:
310             vector = np.zeros(
311                 matrix.shape[0],
312                 dtype=complex
313             )
314
315         eigenvectors.append(vector)
316
317     eigenvectors = np.array(
318         eigenvectors
319     ).T
320
```

```
321     return eigenvalues, eigenvectors
322
323
324 def svd_from_matrix(matrix):
325
326     # A^T
327     transpose = matrix_transpose(matrix)
328
329     # A^T A
330     ata = matrix_product(transpose, matrix)
331
332     # Eigenvalues and eigenvectors of A^T A
333     eigenvalues, eigenvectors = np.linalg.eigh(ata)
334
335     # Sort eigenvalues in descending order
336     order = np.argsort(eigenvalues)[::-1]
337
338     eigenvalues = eigenvalues[order]
339     eigenvectors = eigenvectors[:, order]
340
341     # Remove very small negative values caused by numerical error
342     eigenvalues[eigenvalues < 0] = 0
343
344     # Number of singular values
345     number = min(matrix.shape[0], matrix.shape[1])
346
347     # Singular values
348     singular_values = np.sqrt(
349         eigenvalues[:number]
350     )
351
352     # V
353     V = eigenvectors[:, :number]
354
355     # Sigma
356     sigma = np.zeros(
357         (number, number)
358     )
359
360     for i in range(number):
361         sigma[i, i] = singular_values[i]
362
363     # U = A V / sigma
364     U = np.zeros(
365         (matrix.shape[0], number)
366     )
367
368     for i in range(number):
369
370         if singular_values[i] > 1e-10:
371
372             v = V[:, i].reshape(-1, 1)
373
374             Av = matrix_product(
```

```
375         matrix,
376         v
377     )
378
379     U[:, i] = (
380         Av.flatten()
381         / singular_values[i]
382     )
383
384     return U, sigma, V
385
386
387 def center_data(matrix):
388
389     centered_matrix = matrix.astype(float).copy()
390
391     for j in range(matrix.shape[1]):
392
393         mean = np.mean(matrix[:, j])
394
395         for i in range(matrix.shape[0]):
396
397             centered_matrix[i, j] = (
398                 matrix[i, j] - mean
399             )
400
401     return centered_matrix
402
403
404 def pca(matrix, number_of_components):
405
406     # Center the data
407     centered_matrix = center_data(matrix)
408
409     # SVD of centered data
410     U, sigma, V = svd_from_matrix(
411         centered_matrix
412     )
413
414     # Principal directions
415     principal_directions = V[:, :number_of_components]
416
417     # Project data onto new basis
418     reduced_data = matrix_product(
419         centered_matrix,
420         principal_directions
421     )
422
423     return (
424         centered_matrix,
425         principal_directions,
426         reduced_data
427     )
428
```

```
429
430
431
432 # Part A
433 # Question Specific Functions
434
435 # Q1. Google PageRank
436
437 def pagerank_Q1(matrix):
438
439     eigenvalues, eigenvectors = (
440         eigenvalue_eigenvector(matrix)
441     )
442
443     dominant_index = np.argmax(
444         np.abs(eigenvalues)
445     )
446
447     rank_vector = np.real(
448         eigenvectors[:, dominant_index]
449     )
450
451     rank_vector = np.abs(rank_vector)
452
453     rank_vector = (
454         rank_vector / np.sum(rank_vector)
455     )
456
457     return rank_vector
458
459
460 # Q2. GPS Location Tracking
461
462 def gps_location_Q2(points, distances):
463
464     # Distance equation:
465     #  $(x-a)^2 + (y-b)^2 = d^2$ 
466
467     x1, y1 = points[0]
468     d1 = distances[0]
469
470     coefficient_matrix = []
471     vector_b = []
472
473     for i in range(1, len(points)):
474
475         xi, yi = points[i]
476         di = distances[i]
477
478         coefficient_matrix.append([
479             2 * (xi - x1),
480             2 * (yi - y1)
481         ])
482
```

```
483         vector_b.append(  
484             d1**2  
485             - di**2  
486             + xi**2  
487             + yi**2  
488             - x1**2  
489             - y1**2  
490         )  
491  
492     coefficient_matrix = np.array(  
493         coefficient_matrix,  
494         dtype=float  
495     )  
496  
497     vector_b = np.array(  
498         vector_b,  
499         dtype=float  
500     ).reshape(-1, 1)  
501  
502     position = solve_linear_system(  
503         coefficient_matrix,  
504         vector_b  
505     )  
506  
507     return position  
508  
509  
510 # Q3. Medical Imaging  
511  
512 def medical_imaging_Q3(  
513     matrix_A,  
514     scan_data  
515 ):  
516  
517     # Reconstruct image values by solving A*x = b  
518  
519     image_values = solve_linear_system(  
520         matrix_A,  
521         scan_data  
522     )  
523  
524     return image_values  
525  
526  
527 # Q4. Structural Engineering Vibrations  
528  
529 def Q4_vibration_analysis(stiffness_matrix):  
530  
531     eigenvalues, eigenvectors = eigenvalue_eigenvector(  
532         stiffness_matrix  
533     )  
534  
535     return eigenvalues, eigenvectors  
536
```

```
537
538 # Q5. Wireless Communication (MIMO)
539
540 def mimo_Q5(
541     channel_matrix,
542     received_signal
543 ):
544
545     transmitted_signal = solve_linear_system(
546         channel_matrix,
547         received_signal
548     )
549
550     return transmitted_signal
551
552
553 # Q6. Economics & Market Equilibrium
554
555 def market_equilibrium_Q6(
556     coefficient_matrix,
557     vector_b
558 ):
559
560     equilibrium = solve_linear_system(
561         coefficient_matrix,
562         vector_b
563     )
564
565     return equilibrium
566
567
568 # Q7. Computer Graphics (3D Transformations)
569
570 def graphics_3D_Q7(
571     points,
572     transformation_matrix
573 ):
574
575     transformed_points = matrix_product(
576         transformation_matrix,
577         points
578     )
579
580     return transformed_points
581
582
583 # Q8. Cryptography & Coding Theory
584
585 def cryptography_Q8(
586     key_matrix,
587     message_vector
588 ):
589
590     encoded_message = matrix_product(
```

```
591         key_matrix,
592         message_vector
593     )
594
595     decoded_message = solve_linear_system(
596         key_matrix,
597         encoded_message
598     )
599
600     return encoded_message, decoded_message
601
602
603 # Q9. Epidemiology
604
605 def epidemiology_Q9(
606     transition_matrix,
607     initial_population,
608     steps
609 ):
610
611     population = initial_population.copy()
612
613     for i in range(steps):
614
615         population = matrix_product(
616             transition_matrix,
617             population
618         )
619
620     return population
621
622
623 # Q10. Air Traffic Control
624
625 def air_traffic_control_Q10(
626     coefficient_matrix,
627     measurements
628 ):
629
630     position = solve_linear_system(
631         coefficient_matrix,
632         measurements
633     )
634
635     return position
636
637 # Q11 - Singular Value Decomposition
638
639 def SVD_Q11(matrix):
640
641     U, sigma, V = svd_from_matrix(matrix)
642
643     return U, sigma, V
644
```

```
645
646 # Q12 - Recommender Systems
647
648 def recommender_Q12(rating_matrix):
649
650     U, sigma, V = svd_from_matrix(
651         rating_matrix
652     )
653
654     # Use all available singular values
655     predicted_matrix = matrix_product(
656         matrix_product(U, sigma),
657         matrix_transpose(V)
658     )
659
660     return U, sigma, V, predicted_matrix
661
662
663
664 # Q13 - Feature Spaces / PCA
665
666 def feature_space_Q13(matrix, number_of_components):
667
668     centered_matrix, principal_directions, reduced_data = pca(
669         matrix,
670         number_of_components
671     )
672
673     return (
674         centered_matrix,
675         principal_directions,
676         reduced_data
677     )
678
679
680
681
682
683 ## Execution
684
685
686 # Q1
687 print("Q1.")
688
689 matrix_Q1 = np.array([
690     [0, 0.3, 1],
691     [0.5, 0, 0],
692     [0.5, 0.7, 0]
693 ], dtype=float)
694
695 pagerank = pagerank_Q1(matrix_Q1)
696
697 print("For the given web rank matrix:")
698 print(matrix_Q1)
```

```
699
700 print("PageRank=", pagerank)
701
702
703 # Q2
704 print("\nQ2.")
705
706 points_Q2 = np.array([
707     [0, 0],
708     [4, 0],
709     [0, 4]
710 ], dtype=float)
711
712 distances_Q2 = np.array([
713     5,
714     3,
715     np.sqrt(5)
716 ])
717
718 position_Q2 = gps_location_Q2(
719     points_Q2,
720     distances_Q2
721 )
722
723 print("For the given satellite points:")
724 print(points_Q2)
725 print("and corresponding distances:")
726 print(distances_Q2)
727
728 if position_Q2 is not None:
729     print(
730         "Estimated position=",
731         position_Q2.flatten()
732     )
733
734
735 # Q3
736 print("\nQ3.")
737
738 matrix_Q3 = np.array([
739     [1, 1],
740     [1, -1]
741 ], dtype=float)
742
743 scan_data_Q3 = np.array([
744     [5],
745     [1]
746 ], dtype=float)
747
748 image_Q3 = medical_imaging_Q3(
749     matrix_Q3,
750     scan_data_Q3
751 )
752
```

```
753 print("For the given medical imaging matrix:")
754 print(matrix_Q3)
755 print("and scan data:")
756 print(scan_data_Q3)
757
758 if image_Q3 is not None:
759     print(
760         "Reconstructed values=",
761         image_Q3.flatten()
762     )
763
764
765 # Q4
766
767 stiffness_matrix_Q4 = np.array([
768     [2, -3, 0],
769     [2, -5, 0],
770     [0, 0, 3]
771 ], dtype=float)
772
773 print("\nQ4: Structural Engineering Vibrations")
774
775 print("For the given stiffness matrix:")
776 print(stiffness_matrix_Q4)
777
778 eigenvalues_Q4, eigenvectors_Q4 = Q4_vibration_analysis(
779     stiffness_matrix_Q4
780 )
781
782 print("\nEigenvalues:")
783 print(eigenvalues_Q4)
784
785 print("\nEigenvectors:")
786 print(eigenvectors_Q4)
787
788
789 # Q5
790 print("\nQ5.")
791
792 channel_Q5 = np.array([
793     [2, 0],
794     [0, 3]
795 ], dtype=float)
796
797 received_Q5 = np.array([
798     [2],
799     [3]
800 ], dtype=float)
801
802 transmitted_Q5 = mimo_Q5(
803     channel_Q5,
804     received_Q5
805 )
806
```

```
807 print("For the given communication channel matrix:")
808 print(channel_Q5)
809 print("and received signal:")
810 print(received_Q5)
811
812 if transmitted_Q5 is not None:
813     print(
814         "Recovered signal=",
815         transmitted_Q5.flatten()
816     )
817
818
819 # Q6
820 print("\nQ6.")
821
822 coefficient_Q6 = np.array([
823     [1, 1],
824     [-1, 1]
825 ], dtype=float)
826
827 vector_Q6 = np.array([
828     [10],
829     [2]
830 ], dtype=float)
831
832 equilibrium_Q6 = market_equilibrium_Q6(
833     coefficient_Q6,
834     vector_Q6
835 )
836
837 print("For the given market coefficient matrix:")
838 print(coefficient_Q6)
839 print("and demand/supply vector:")
840 print(vector_Q6)
841
842 if equilibrium_Q6 is not None:
843     print(
844         "Equilibrium=",
845         equilibrium_Q6.flatten()
846     )
847
848
849 # Q7
850 print("\nQ7.")
851
852 points_Q7 = np.array([
853     [1, 0],
854     [0, 1]
855 ], dtype=float)
856
857 rotation_Q7 = np.array([
858     [0, -1],
859     [1, 0]
860 ], dtype=float)
```

```
861
862 transformed_Q7 = graphics_3D_Q7(
863     points_Q7,
864     rotation_Q7
865 )
866
867 print("For the given points:")
868 print(points_Q7)
869 print("and transformation matrix:")
870 print(rotation_Q7)
871
872 print(
873     "Transformed points=\n",
874     transformed_Q7
875 )
876
877
878 # Q8
879 print("\nQ8.")
880
881 key_Q8 = np.array([
882     [1, 1],
883     [0, 1]
884 ], dtype=float)
885
886 message_Q8 = np.array([
887     [1],
888     [2]
889 ], dtype=float)
890
891 encoded_Q8, decoded_Q8 = (
892     cryptography_Q8(
893         key_Q8,
894         message_Q8
895     )
896 )
897 print("For the given key matrix:")
898 print(key_Q8)
899 print("and message vector:")
900 print(message_Q8)
901
902 print(
903     "Encoded message=",
904     encoded_Q8.flatten()
905 )
906
907 print(
908     "Decoded message=",
909     decoded_Q8.flatten()
910 )
911
912
913 # Q9
914 print("\nQ9.")
```

```
915
916 transition_Q9 = np.array([
917     [0.9, 0.1],
918     [0.1, 0.9]
919 ], dtype=float)
920
921 initial_population_Q9 = np.array([
922     [90],
923     [10]
924 ], dtype=float)
925
926 population_Q9 = epidemiology_Q9(
927     transition_Q9,
928     initial_population_Q9,
929     1
930 )
931
932 print("For the given epidemiological transition matrix:")
933 print(transition_Q9)
934 print("and initial population:")
935 print(initial_population_Q9)
936
937 print(
938     "Population after one step=",
939     population_Q9.flatten()
940 )
941
942
943 # Q10
944 print("\nQ10.")
945
946 coefficient_Q10 = np.array([
947     [1, 1],
948     [1, -1]
949 ], dtype=float)
950
951 measurements_Q10 = np.array([
952     [10],
953     [2]
954 ], dtype=float)
955
956 position_Q10 = air_traffic_control_Q10(
957     coefficient_Q10,
958     measurements_Q10
959 )
960
961 print("For the given aircraft measurement coefficient matrix:")
962 print(coefficient_Q10)
963 print("and measurements:")
964 print(measurements_Q10)
965
966 if position_Q10 is not None:
967     print(
968         "Aircraft position=",
```

```
969         position_Q10.flatten()
970     )
971
972
973     # Q11
974
975     matrix_M1 = np.array([
976         [1, -1],
977         [0, 1],
978         [1, 0]
979     ], dtype=float)
980
981     matrix_M2 = np.array([
982         [1, 1, 1],
983         [-1, 0, -2],
984         [1, 2, 0]
985     ], dtype=float)
986
987
988     print("\nQ11: Singular Value Decomposition")
989
990     print("\nFor the given matrix M1:")
991     print(matrix_M1)
992
993     U1, Sigma1, V1 = SVD_Q11(matrix_M1)
994
995     print("\nU for M1:")
996     print(U1)
997
998     print("\nSigma for M1:")
999     print(Sigma1)
1000
1001     print("\nV^T for M1:")
1002     print(matrix_transpose(V1))
1003
1004
1005     print("\nFor the given matrix M2:")
1006     print(matrix_M2)
1007
1008     U2, Sigma2, V2 = SVD_Q11(matrix_M2)
1009
1010     print("\nU for M2:")
1011     print(U2)
1012
1013     print("\nSigma for M2:")
1014     print(Sigma2)
1015
1016     print("\nV^T for M2:")
1017     print(matrix_transpose(V2))
1018
1019
1020     # Q12
1021
1022     rating_matrix_Q12 = np.array([
```

```
1023     [5, 4, 0, 1],
1024     [4, 0, 2, 1],
1025     [1, 2, 5, 4]
1026 ], dtype=float)
1027
1028
1029 print("\nQ12: Recommender Systems")
1030
1031 print("\nFor the given user-item rating matrix:")
1032 print(rating_matrix_Q12)
1033
1034 U12, Sigma12, V12, predicted_Q12 = recommender_Q12(
1035     rating_matrix_Q12
1036 )
1037
1038 print("\nU:")
1039 print(U12)
1040
1041 print("\nSigma:")
1042 print(Sigma12)
1043
1044 print("\nV^T:")
1045 print(matrix_transpose(V12))
1046
1047 print("\nPredicted/reconstructed rating matrix:")
1048 print(predicted_Q12)
1049
1050
1051 # Q13
1052
1053 data_matrix_Q13 = np.array([
1054     [2, 1],
1055     [3, 2],
1056     [4, 3],
1057     [5, 4]
1058 ], dtype=float)
1059
1060
1061 print("\nQ13: Machine Learning - Feature Spaces")
1062
1063 print("\nFor the given feature matrix:")
1064 print(data_matrix_Q13)
1065
1066 centered_Q13, basis_Q13, reduced_Q13 = feature_space_Q13(
1067     data_matrix_Q13,
1068     1
1069 )
1070
1071 print("\nCentered data:")
1072 print(centered_Q13)
1073
1074 print("\nPrincipal basis direction:")
1075 print(basis_Q13)
1076
```

```
1077 print("\nReduced one-dimensional representation:")
1078 print(reduced_Q13)
1079
1080
1081
1082
1083
1084 '''
1085 Q1.
1086 For the given web rank matrix:
1087 [[0.  0.3 1. ]
1088  [0.5 0.  0. ]
1089  [0.5 0.7 0. ]]
1090 PageRank= [0.75 0.25 0.  ]
1091
1092 Q2.
1093 For the given satellite points:
1094 [[0. 0.]
1095  [4. 0.]
1096  [0. 4.]]
1097 and corresponding distances:
1098 [5.          3.          2.23606798]
1099 Estimated position= [4.  4.5]
1100
1101 Q3.
1102 For the given medical imaging matrix:
1103 [[ 1.  1.]
1104  [ 1. -1.]]
1105 and scan data:
1106 [[5.]
1107  [1.]]
1108 Reconstructed values= [3. 2.]
1109
1110 Q4: Structural Engineering Vibrations
1111 For the given stiffness matrix:
1112 [[ 2. -3.  0.]
1113  [ 2. -5.  0.]
1114  [ 0.  0.  3.]]
1115
1116 Eigenvalues:
1117 [-4.  3.  1.]
1118
1119 Eigenvectors:
1120 [[1.+0.j 0.+0.j 3.+0.j]
1121  [2.+0.j 0.+0.j 1.+0.j]
1122  [0.+0.j 1.+0.j 0.+0.j]]
1123
1124 Q5.
1125 For the given communication channel matrix:
1126 [[2. 0.]
1127  [0. 3.]]
1128 and received signal:
1129 [[2.]
1130  [3.]]
```

```
1131 Recovered signal= [1. 1.]
1132
1133 Q6.
1134 For the given market coefficient matrix:
1135 [[ 1.  1.]
1136  [-1.  1.]]
1137 and demand/supply vector:
1138 [[10.]
1139  [ 2.]]
1140 Equilibrium= [4. 6.]
1141
1142 Q7.
1143 For the given points:
1144 [[1. 0.]
1145  [0. 1.]]
1146 and transformation matrix:
1147 [[ 0. -1.]
1148  [ 1.  0.]]
1149 Transformed points=
1150 [[ 0. -1.]
1151  [ 1.  0.]]
1152
1153 Q8.
1154 For the given key matrix:
1155 [[1. 1.]
1156  [0. 1.]]
1157 and message vector:
1158 [[1.]
1159  [2.]]
1160 Encoded message= [3. 2.]
1161 Decoded message= [1. 2.]
1162
1163 Q9.
1164 For the given epidemiological transition matrix:
1165 [[0.9 0.1]
1166  [0.1 0.9]]
1167 and initial population:
1168 [[90.]
1169  [10.]]
1170 Population after one step= [82. 18.]
1171
1172 Q10.
1173 For the given aircraft measurement coefficient matrix:
1174 [[ 1.  1.]
1175  [ 1. -1.]]
1176 and measurements:
1177 [[10.]
1178  [ 2.]]
1179 Aircraft position= [6. 4.]
1180
1181 Q11: Singular Value Decomposition
1182
1183 For the given matrix M1:
1184 [[ 1. -1.]
```

```
1185     [ 0.  1.]
1186     [ 1.  0.]]
1187
1188     U for M1:
1189     [[-0.81649658  0.
1190      [ 0.40824829 -0.70710678]
1191      [-0.40824829 -0.70710678]]
1192
1193     Sigma for M1:
1194     [[1.73205081 0.
1195      [0. 1.
1196
1197     V^T for M1:
1198     [[-0.70710678  0.70710678]
1199      [-0.70710678 -0.70710678]]
1200
1201     For the given matrix M2:
1202     [[ 1.  1.  1.]
1203      [-1.  0. -2.]
1204      [ 1.  2.  0.]]
1205
1206     U for M2:
1207     [[-5.77350269e-01  5.55111512e-17  0.00000000e+00]
1208      [ 5.77350269e-01 -7.07106781e-01  0.00000000e+00]
1209      [-5.77350269e-01 -7.07106781e-01  0.00000000e+00]]
1210
1211     Sigma for M2:
1212     [[3. 0. 0.]
1213      [0. 2. 0.]
1214      [0. 0. 0.]]
1215
1216     V^T for M2:
1217     [[-0.57735027 -0.57735027 -0.57735027]
1218      [ 0. -0.70710678  0.70710678]
1219      [-0.81649658  0.40824829  0.40824829]]
1220
1221     Q12: Recommender Systems
1222
1223     For the given user-item rating matrix:
1224     [[5. 4. 0. 1.]
1225      [4. 0. 2. 1.]
1226      [1. 2. 5. 4.]]
1227
1228     U:
1229     [[-0.61797905 -0.66758089  0.41525612]
1230      [-0.45368144 -0.1285607  -0.88184199]
1231      [-0.64208648  0.73335387  0.22342124]]
1232
1233     Sigma:
1234     [[8.66487473 0. 0.
1235      [0. 5.23155829 0.
1236      [0. 0. 2.55944208]]
1237
1238     V^T:
```

```
1239 [[-0.64013706 -0.43348453 -0.47522848 -0.4200876 ]
1240 [-0.59614998 -0.23006832  0.65174615  0.40853485]
1241 [-0.47966161  0.82356503 -0.2526245   0.16687195]]
1242
1243 Predicted/reconstructed rating matrix:
1244 [[5.00000000e+00 4.00000000e+00 1.11022302e-15 1.00000000e+00]
1245 [4.00000000e+00 2.22044605e-16 2.00000000e+00 1.00000000e+00]
1246 [1.00000000e+00 2.00000000e+00 5.00000000e+00 4.00000000e+00]]
1247
1248 Q13: Machine Learning - Feature Spaces
1249
1250 For the given feature matrix:
1251 [[2. 1.]
1252 [3. 2.]
1253 [4. 3.]
1254 [5. 4.]]
1255
1256 Centered data:
1257 [[-1.5 -1.5]
1258 [-0.5 -0.5]
1259 [ 0.5  0.5]
1260 [ 1.5  1.5]]
1261
1262 Principal basis direction:
1263 [[0.70710678]
1264 [0.70710678]]
1265
1266 Reduced one-dimensional representation:
1267 [[-2.12132034]
1268 [-0.70710678]
1269 [ 0.70710678]
1270 [ 2.12132034]]
1271 '''
```